AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

NAME: RAAGHAV VEL P

CSE-B

CH.SC.U4CSE24155

1) Bubble Sort:

CODE:

```
#include <stdio.h>

void main() {
    int a[50], n, i, j, t;
    scanf("%d", &n);
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]) {
                t=a[j];
                a[j]=a[j+1];
                a[j+1]=t;
            }
    for(i=0;i<n;i++) printf("%d ",a[i]);
}
```

OUTPUT:

```
C:\II year\4th sem\DAA\L
8
3 5 4 2 7 5 6 3
2 3 3 4 5 5 6 7
```

JUSTIFICATION:

Bubble sort compares adjacent elements repeatedly until the list is sorted. In the worst and average cases, it uses two nested loops, resulting in a time complexity of $O(n^2)$. In the best case, when the array is already sorted, the time complexity becomes $O(n)$. The space complexity is $O(1)$ because sorting is done in-place using only a constant amount of extra memory for swapping variables.

2) Insertion Sort

CODE:

```
#include <stdio.h>

void main() {
    int a[50], n, i, j, key;
    scanf("%d",&n);
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    for(i=1;i<n;i++) {
        key=a[i];
        j=i-1;
        while(j>=0 && a[j]>key) {
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
    for(i=0;i<n;i++) printf("%d ",a[i]);
}
```

OUTPUT:

```
5
4 3 2 5 1
1 2 3 4 5
C:\II year\4th se
```

JUSTIFICATION:

Insertion sort builds the sorted array one element at a time by inserting each element into its correct position. In the worst and average cases, each element may need to be compared with all previous elements, giving a time complexity of $O(n^2)$. In the best case, when the array is already sorted, the time complexity is $O(n)$. The space complexity is $O(1)$ since no additional data structures are used and sorting is performed in-place.

3) Selection Sort:

CODE:

```
#include <stdio.h>

void main() {
    int a[50], n, i, j, min, t;
    scanf("%d",&n);
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    for(i=0;i<n-1;i++) {
        min=i;
        for(j=i+1;j<n;j++)
            if(a[j]<a[min]) min=j;
        t=a[i];
        a[i]=a[min];
        a[min]=t;
    }
    for(i=0;i<n;i++) printf("%d ",a[i]);
}
```

OUTPUT:

```
C:\II year\4th sem\
7
4 3 6 2 7 8 4
2 3 4 4 6 7 8
```

JUSTIFICATION:

Selection sort repeatedly selects the smallest element from the unsorted portion and places it in the correct position. It always uses two nested loops regardless of input order, so its time complexity is $O(n^2)$ in all cases. The space complexity is $O(1)$ because only a temporary variable is used for swapping, making it an in-place sorting algorithm.

4) Bucket Sort:

CODE:

```
#include <stdio.h>

void main() {
    int a[50], bucket[50]={0}, n, i;
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        scanf("%d",&a[i]);
        bucket[a[i]]++;
    }
    for(i=0;i<50;i++)
        while(bucket[i]--)
            printf("%d ",i);
}
```

OUTPUT:

```
C:\II year\4th sem\D
6
5 3 4 3 2 1
1 2 3 3 4 5
```

JUSTIFICATION:

Bucket sort distributes elements into buckets and then sorts each bucket individually. The average time complexity is O(n + k), where k is the number of buckets, assuming uniform distribution of elements. The worst-case time complexity can be $O(n^2)$ if all elements fall into a single bucket. The space complexity is O(n + k) because additional memory is required to store the buckets apart from the original array.

5) Heap Sort:

CODE:

```
#include <stdio.h>

void heapify(int a[], int n, int i) {
    int largest=i,l=2*i+1,r=2*i+2,t;
    if(l<n && a[l]>a[largest]) largest=l;
    if(r<n && a[r]>a[largest]) largest=r;
    if(largest!=i) {
        t=a[i]; a[i]=a[largest]; a[largest]=t;
        heapify(a,n,largest);
    }
}

void main() {
    int a[50], n, i, t;
    scanf("%d",&n);
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    for(i=n/2-1;i>=0;i--) heapify(a,n,i);
    for(i=n-1;i>0;i--) {
        t=a[0]; a[0]=a[i]; a[i]=t;
        heapify(a,i,0);
    }
    for(i=0;i<n;i++) printf("%d ",a[i]);
}
```

OUTPUT:

```
C:\II year\4th sem\
6
2 4 3 5 3 1
1 2 3 3 4 5
```

JUSTIFICATION:

Heap sort builds a heap data structure and repeatedly extracts the maximum or minimum element. Building the heap takes O(n) time, and each extraction takes O(log n), resulting in an overall time complexity of O(n log n) in all cases. The space complexity is O(1) because heap sort rearranges elements within the original array and does not require extra memory.

6) BFS:

CODE:

```
#include <stdio.h>

void main() {
    int n,i,j,s,q[50],f=0,r=0;
    int a[50][50],v[50]={0};
    scanf("%d",&n);
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&a[i][j]);
    scanf("%d",&s);
    q[r++]=s;
    v[s]=1;
    while(f<r) {
        s=q[f++];
        printf("%d ",s);
        for(i=0;i<n;i++)
            if(a[s][i]==1 && v[i]==0) {
                q[r++]=i;
                v[i]=1;
            }
    }
}
```

OUTPUT:

```
C:\II year\4th sem\D
8
5 4 7 3 2 9 7 5
2 3 4 5 5 7 7 9
```

JUSTIFICATION:

BFS explores all vertices level by level using a queue. The time complexity is O(V + E), where V is the number of vertices and E is the number of edges, since each vertex and edge is visited once. The space complexity is O(V) because additional memory is required to store the queue and the visited array.

7) DFS:

CODE:

```
#include <stdio.h>

int a[50][50], v[50]={0}, n;

void dfs(int s) {
    int i;
    printf("%d ",s);
    v[s]=1;
    for(i=0;i<n;i++)
        if(a[s][i]==1 && v[i]==0)
            dfs(i);
}

void main() {
    int i,j,s;
    scanf("%d",&n);
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&a[i][j]);
    scanf("%d",&s);
    dfs(s);
}
```

OUTPUT:

```
C:\II year\4th sem\DAA\LAB
10
5 4 7 6 2 3 10 9 8 1
1 2 3 4 5 6 7 8 9 10
```

JUSTIFICATION:

DFS explores vertices by going as deep as possible before backtracking. Its time complexity is O(V + E) because all vertices and edges are visited once. The space complexity is O(V) due to the recursive call stack (or explicit stack) and the visited array used during traversal.